

Practical Work 4: Word Count

Nguyen Trong Bach
Student ID: 23BI14057

December 6, 2025

1 Introduction

This practical implements the classical Word Count example using a simple MapReduce style framework written in C++. Instead of using a full MapReduce system such as Hadoop, I design a minimal implementation with clearly separated **Mapper** and **Reducer** functions. The goal is to count how many times each word appears in an input text file.

2 Choice of Implementation

C++ is chosen for this practical because:

- it is close to C and fits well with the previous practical works;
- the Standard Library provides convenient containers such as `std::map` to store key-value pairs;
- the program can be compiled and executed easily on the course environment using `g++`.

3 Mapper and Reducer Design

Mapper

The Mapper is implemented by the function `map_line(const std::string&, std::map<std::string,int>&)`. For each input line the Mapper:

1. scans the line character by character;
2. groups sequences of alphabetic characters into words;
3. converts each word to lowercase to avoid case differences;
4. increases the local counter for that word in a local map of type `std::map<std::string,int>`.

In logical MapReduce terms, for every word the pair (`word`, 1) is produced and stored in the local map.

Reducer

The Reducer is implemented by the function `reduce_counts(const std::map<std::string,int>&, std::map<std::string,int>&)`. The Reducer receives a local map containing pairs of the form (`word`, `count`) and merges them into a global map:

- for each key in the local map the corresponding value is added to the value stored in the global map;

- if the key does not exist in the global map yet, it is inserted with the given count.

This behaviour is similar to the Reduce phase in MapReduce, where all values associated with the same key are combined.

4 System Organization

The whole Word Count system is contained in a single source file `word_count.cpp`. The main program follows these steps:

1. read the input file name and output file name from the command line arguments;
2. open the input file and read it line by line;
3. for each line, call the Mapper to obtain local word counts;
4. call the Reducer to merge the local counts into a global map;
5. after all lines have been processed, write the global word counts to the output file, one word per line.

The output format is very simple: each line contains a word followed by its total count, for example:

```
apple 5
banana 3
```

5 Implementation Snippet

The following listing shows the core part of the Mapper function:

```
void map_line(const std::string &line ,
              std::map<std::string , int> &local_counts) {
    std::string word;
    for (char c : line) {
        if (std::isalpha(static_cast<unsigned char>(c))) {
            word.push_back(
                static_cast<char>(std::tolower(
                    static_cast<unsigned char>(c))));
        } else {
            if (!word.empty()) {
                ++local_counts[word];
                word.clear();
            }
        }
    }
    if (!word.empty()) {
        ++local_counts[word];
    }
}
```

The Reducer simply iterates over the local map and adds all counts to the global map.

6 Group Work

This practical work was completed individually. I was responsible for designing the simple MapReduce style, implementing the C++ program and writing this report.

7 Conclusion

This practical demonstrates that the MapReduce programming model can be implemented even without a full framework. By separating the logic into Mapper and Reducer functions, the program mirrors the core ideas of MapReduce while remaining small and easy to understand. The same design could later be extended to run in parallel using multiple processes or machines.